

Table des matières

Les techniques de transfert de données	2
Les sessions	2
Principe de fonctionnement des sessions	2
Application pratique	3
<i>Principe</i>	<i>3</i>
<i>La page de connexion</i>	<i>3</i>
<i>La page de traitement du formulaire</i>	<i>4</i>
<i>La page membre</i>	<i>5</i>
<i>La page de déconnexion</i>	<i>5</i>

Les techniques de transfert de données

Pour transmettre des variables de pages en pages, plusieurs techniques existent :

- Utiliser les **champs des formulaires**
- Passer les **variables** à travers les **liens**
- Utiliser les **cookies**
- Utiliser les **sessions**

Chaque **technique** est spécifique à un **mode d'utilisation** et n'offre pas le même **niveau de sécurité** :

- Tous les utilisateurs n'acceptent pas les **cookies**
- Le passage des **variables** par les liens offre un **bas niveau de sécurité**, les variables étant visibles de tout le monde
- Dans une page de **formulaire**, utiliser un **champ** de type **hidden** pour faire circuler des différentes variables à travers toutes les pages du site est peu pratique

Les sessions

Les **sessions** offrent la technique la plus **pratique** et la plus **sécurisée** de faire circuler différentes variables (comme un mot de passe par exemple) à travers les pages d'un site.

Pour utiliser les **sessions**, différentes **fonctions PHP** existent :

Fonction	Signification
<code>session_start</code>	Démarre une session
<code>session_register</code>	Enregistre une variable de session
<code>session_unregister</code>	Efface une variable de session
<code>session_is_registered</code>	Vérifie si une variable est déclarée pour la session en cours
<code>session_id</code>	Retourne l'id de la session en cours
<code>session_name</code>	Retourne le nom de la session en cours
<code>session_unset</code>	Detruit toutes les variables de la session en cours
<code>session_destroy</code>	Detruit la session en cours

Les sessions sont accessibles depuis **PHP 4...**

Principe de fonctionnement des sessions

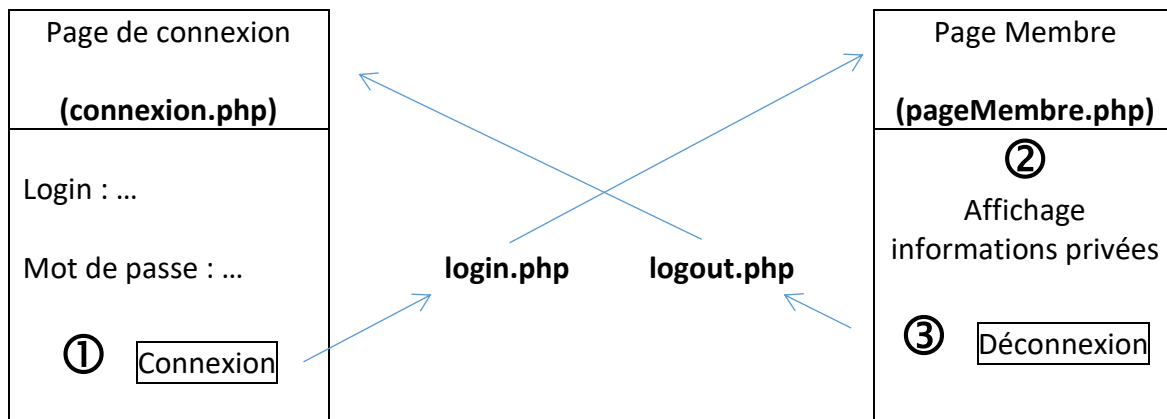
- Chaque session possède un **id différent**, ce qui permet d'éviter la confusion entre les connexions.
- À chaque page où la session doit être active, il faut placer un **session_start** en tout début de page, **avant tout code HTML**.
- Toutes les **variables enregistrées** au cours d'une session seront **accessibles dans les pages de la session**.
- Ne **JAMAIS** oublier de **détruire les variables de session** lors de la déconnexion.

Application pratique

Principe

Création d'une **page réservée aux membres** :

1. L'utilisateur est invité à se connecter avec un **identifiant** et un **mot de passe**
2. Si les informations de connexion sont correctes, la **page membre** s'affiche
3. Sur cette page membre, un bouton de **déconnexion** permet de quitter la session et de réafficher la page de connexion



L'ensemble des fichiers sera placé dans un **dossier Session** dans le **www** de **Wamp**.

La page de connexion

Cette page contient un formulaire permettant de se connecter à une section membre.

Page **connexion.php** :

```
<html>
  <head>
    <title>Formulaire d'identification</title>
  </head>
  <body>
    <form action="login.php" method="post">
      Votre login : <input type="text" name="login">
      <br />
      Votre mot de passe : <input type="password" name="pwd"><br />
      <input type="submit" value="Connexion">
    </form>
  </body>
</html>
```

Cette page appelle, au clic sur le bouton Connexion, la page **login.php**.

La page de traitement du formulaire

Fonctionnement de cette page **login.php** :

- Récupération des variables postées du formulaire : `$_POST['login']` et `$_POST['pwd']`
- Si ces **données** sont **valides** :
 - Démarre de la session avec **session_start ()**
 - Création de la variable de session : `$_SESSION['login'] = $_POST['login'];`
 - Affichage de la page membre : **header ('location: pageMembre.php') ;**
- Si ces **données** sont **invalides** :
 - **Affichage** d'une information dans une fenêtre **popup**
 - **Redirection** vers la **page de connexion**

```
<?php
// On définit un login et un mot de passe de base pour tester notre exemple.
$login_valide = "test";
$pwd_valide = "pwd";

// on teste si nos variables sont définies
if (isset($_POST['login']) && isset($_POST['pwd'])) {

    // on vérifie les informations du formulaire, à savoir si le pseudo saisi est bien un pseudo
    // autorisé, de même pour le mot de passe
    if ($login_valide == $_POST['login'] && $pwd_valide == $_POST['pwd']) {
        // dans ce cas, tout est ok, on peut démarrer notre session

        // on la démarre
        session_start ();
        // on enregistre le paramètre de notre visiteur en variable de session ($login)
        $_SESSION['login'] = $_POST['login'];

        // on redirige notre visiteur vers une page de notre section membre
        header ('location: pageMembre.php');
    }
    else {
        // Si les données d'identification sont incorrectes, on affiche une popup
        echo '<body onLoad="alert(\'Membre non reconnu...\')">';
        // puis on le redirige vers la page de connexion
        echo '<meta http-equiv="refresh" content="0;URL=connexion.php">';
    }
}
else {
    echo 'Les variables du formulaire ne sont pas déclarées.';
}
?>
```

Remarque : Le **session_start** doit figurer avant tout code HTML.

La page membre

Fonctionnement de cette page **pageMembre.php** :

- Démarre la session **session_start ()**;
- **Récupère** et **teste** les **variables de session**.
- Si les variables sont **valides**, elle affiche le **contenu** de la page.
- Propose un bouton de **déconnexion** de la session

```
<?php
// On démarre la session (à placer dans toutes les pages de la section membre)
session_start ();

// On récupère nos variables de session
if (isset($_SESSION['login'])) {

    // On teste pour voir si nos variables ont bien été enregistrées
    echo '<html>';
    echo '<head>';
    echo '<title>Page de notre section membre</title>';
    echo '</head>';

    echo '<body>';
    echo 'Bonjour : ' . $_SESSION['login'] . '. Vous êtes sur une page membre';

    echo '<br />';

    // On affiche un lien pour fermer notre session
    echo '<a href= "/>
```

La page de déconnexion

Cette page **logout.php** annule la session et ses variables pour une nouvelle connexion.

```
<?php
// On démarre la session
session_start ();
// On détruit les variables de notre session
session_unset ();
// On détruit notre session
session_destroy ();
// On redirige le visiteur vers la page d'accueil
header ('location: connexion.php');
?>
```

Amélioration de la connexion

Dans l'exercice ci-dessus, par souci de simplification, le **login** et le mot de passe sont enregistré dans des **variables** du programme (`$login_valide = "test"; $pwd_valide = "pwd";`).

Améliorer cette solution en intégrant les points suivants :

- **Enregistrer** un **login** et un **mot de passe** dans la table **personne**
- Utiliser un **mot de passe haché**
- **Vérifier** lors de la **connexion** à l'aide du **formulaire** si les informations d'**identification** sont **valides**, c'est-à-dire si le login et le mot de passe saisis dans le formulaire sont conformes à ceux présents dans la table.